

# 1 Solution — GIAM 2.2

## Problem 7

### 1.1 Problem

Express the statement  $A \Rightarrow B$  using the Peirce arrow ( $\downarrow$ , NOR) and/or the Scheffer stroke ( $|$ , NAND). (Recall Problem 2.1.5, where  $A | B = \neg(A \wedge B)$  and  $A \downarrow B = \neg(A \vee B)$ .)

### 1.2 Answer

The shortest form uses the Scheffer stroke alone:

$$A \Rightarrow B = A | (B | B).$$

Using the Peirce arrow alone:

$$A \Rightarrow B = ((A \downarrow A) \downarrow B) \downarrow ((A \downarrow A) \downarrow B).$$

### 1.3 Derivation

Start from the material-implication rewrite  $A \Rightarrow B \equiv \neg A \vee B$  (Problem 2), and recall from Problem 2.1.5 that each connective builds negation from itself:  $\neg X = X | X = X \downarrow X$ .

**Scheffer stroke (NAND).** Rewrite the disjunction as a NAND via De Morgan:

$$A \Rightarrow B \equiv \neg A \vee B \equiv \neg(A \wedge \neg B) = A | \neg B.$$

Then replace  $\neg B$  by  $B | B$ :

$$A \Rightarrow B = A | (B | B).$$

**Peirce arrow (NOR).** Write the disjunction as a doubly-negated NOR:

$$A \Rightarrow B \equiv \neg A \vee B = \neg(\neg A \downarrow B) = (\neg A \downarrow B) \downarrow (\neg A \downarrow B),$$

then substitute  $\neg A = A \downarrow A$  to get the boxed NOR expression.

*Start from  $A \Rightarrow B \equiv \neg A \vee B$ , then express with the single-gate connectives.*

**Via NAND ( $A | B = \neg(A \wedge B)$ )**

$$A \Rightarrow B \equiv \neg A \vee B \equiv \neg(A \wedge \neg B) = A | \neg B$$

and  $\neg B = B | B$ , so:

$$A \Rightarrow B = A | (B | B)$$

**Via NOR ( $A \downarrow B = \neg(A \vee B)$ )**

$$A \Rightarrow B \equiv \neg A \vee B = \neg(\neg A \downarrow B) \text{ and } \neg A = A \downarrow A, \text{ so:}$$

$$A \Rightarrow B = ((A \downarrow A) \downarrow B) \downarrow ((A \downarrow A) \downarrow B)$$

*(a NOR of a value with itself is negation, undoing the inner NOR)*

**Both reproduce  $A \Rightarrow B$  on all four rows:**

$$A \ B : A \Rightarrow B$$

$$1 \ 1 : 1 \ 1 \ 0 : 0$$

$$0 \ 1 : 1 \ 0 \ 0 : 1$$

$$A | (B | B) \checkmark$$

$$((A \downarrow A) \downarrow B) \downarrow ((A \downarrow A) \downarrow B) \checkmark$$

Figure 1.1: Two derivations. Via NAND,  $A | B = \text{not}(A \text{ and } B)$ :  $A$  implies  $B$  is  $\text{not-}A$  or  $B$ , which is  $\text{not}(A \text{ and } \text{not-}B) = A | \text{not-}B$ ; since  $\text{not-}B = B | B$ , the result is  $A | (B | B)$ . Via NOR,  $A \downarrow B = \text{not}(A \text{ or } B)$ :  $A$  implies  $B$  is  $\text{not-}A$  or  $B = \text{not}(\text{not-}A \downarrow B)$ ; since  $\text{not-}A = A \downarrow A$ , the result is  $((A \downarrow A) \downarrow B) \downarrow ((A \downarrow A) \downarrow B)$ . A small table confirms both reproduce  $A$  implies  $B$  on all four rows: 1 1 gives 1, 1 0 gives 0, 0 1 gives 1, 0 0 gives 1.

## 1.4 Verification of the NAND Form

Tracing  $A | (B | B)$  (recall  $X | Y$  is false only when both  $X, Y$  are true):

| $A$ | $B$ | $B   B (= \neg B)$ | $A   (B   B)$ | $A \Rightarrow B$ |
|-----|-----|--------------------|---------------|-------------------|
| T   | T   | F                  | T             | T                 |
| T   | F   | T                  | F             | F                 |
| F   | T   | F                  | T             | T                 |
| F   | F   | T                  | T             | T                 |

Table 1.1

The computed column matches  $A \Rightarrow B$  in every row. ■

## 1.5 Remark — Why the NAND Form Is So Short

The Scheffer stroke wins here because implication is *almost already* a NAND. Unwinding the definition,  $A \mid \neg B = \neg(A \wedge \neg B)$ , and  $\neg(A \wedge \neg B)$  is exactly “not ( $A$  but not  $B$ )” — which is precisely the conditional (recall  $A \wedge \neg B$  is the *negation* of  $A \Rightarrow B$  from Problem 4!). So  $A \Rightarrow B$  is literally “NAND of  $A$  with  $\neg B$ ,” and the only extra work is manufacturing  $\neg B = B \mid B$ . The result,  $A \mid (B \mid B)$ , uses just **three** strokes, versus **five** arrows for the NOR version — a nice illustration that the choice of primitive connective changes how compactly a given function can be written.

## 1.6 Remark — Connecting to the Negation of Problem 4

This problem and Problem 4 are two sides of one coin. Problem 4 found  $\neg(A \Rightarrow B) = A \wedge \neg B$ . Since the Scheffer stroke *is* negated-and,  $A \mid \neg B = \neg(A \wedge \neg B) = \neg(\neg(A \Rightarrow B)) = A \Rightarrow B$ . In other words, the single NAND gate directly *un-negates* the “counterexample” formula  $A \wedge \neg B$ . Seeing the implication as “the NAND that rejects only the counterexample row” ties the two exercises together: the one row where  $A \wedge \neg B$  is true is exactly the one row where  $A \Rightarrow B$  is false.

## 1.7 Remark — Everything Really Does Reduce to One Gate

This is the payoff of Problem 2.1.5’s functional completeness. There we saw NAND (and NOR) alone can express *any* truth function; here we cash that out for the conditional, the one connective not in the basic  $\{\wedge, \vee, \neg\}$  toolkit. Because  $A \Rightarrow B$  reduces to strokes, a circuit built entirely from NAND gates can compute implication — feed  $B$  into a NAND with both inputs tied (making  $\neg B$ ), then NAND that with  $A$ . Two physical gates realise “ $A \Rightarrow B$ .” This is why a chip fabricator needs only master a single gate

type to build control logic full of conditionals, and it closes the loop with the hardware theme running through this chapter's problems.

## 1.8 Remark — The Forms Are Not Unique

Several equally-valid expressions exist, since the algebra of  $|$  and  $\downarrow$  offers many routes. For instance, one could expand  $\neg A \vee B$  differently and reach a longer NAND expression, or mix the two connectives. The boxed answers are simply the *shortest* in each single-connective style. What is forced is only the *function* computed (the truth table), not the syntactic form — a reminder that “expressing” a statement in a given basis admits many correct answers, and minimality is a separate (often harder) optimisation question, central to real circuit design.